e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

AN INTEGRATED EVENT MANAGEMENT AND TICKET BOOKING SYSTEM

Mrs. Runal Pruthviraj Pawar*1, Rugved Dilip Adkar*2, Aditya Laxman Rathod*3, Sohamm Prasad Kulkarni*4, Aniket Sanjay Kawadkar*5

*1Assistant Professor, Department Of Electrical And Computer Engineering, Bharati Vidyapeeth (Deemed To Be University), Pune, Maharashtra, India.

*2,3,4,5B.Tech. Electrical And Computer Engineering, Department Of Electrical And Computer Engineering, Bharati Vidyapeeth (Deemed To Be University), Pune, Maharashtra, India.

DOI: https://www.doi.org/10.56726/IRJMETS102890

ABSTRACT

The following recurring issues need to be addressed by online ticket sales services: numerous customers trying to buy the last remaining seats simultaneously, fragmented event data stored in different applications, and tickets that can be easily duplicated. This paper presents an Event Management System (EMS) able to address all these problems in one software solution. The system architecture consists of a three-tier system with a browser client, stateless REST API, and a relational database in third normal form. Allocation of tickets happens within a serializable transaction using row-level locks, thus preventing simultaneous requests from selling out more tickets than are actually available. Each ticket is signed with a QR-code that can be validated just once, and roles based authentication ensures that functions of Guest, Organizer, and Administrator do not overlap. The system was thoroughly tested using functional test cases and load testing with multiple users. In five test runs it achieved a mean response time of 142 ms for the API, remained stable up to 750 simulated users, did not oversell any tickets during 50,000 booking requests, and validated tickets in 185 ms on average.

Keywords: Event Management System, Ticket Booking, Concurrency Control, QR Authentication, Role-Based Access Control, RESTful Architecture.

I. INTRODUCTION

Internet-based purchases of tickets have become commonplace, but many reservation systems continue to encounter problems during peak demand time. Whether a department making 150 seats available for a symposium or a festival issuing 20,000 tickets in ten minutes, there is a similar technological issue in each case. Many users are trying to access a limited number of seats which keeps decreasing, while the database has to reflect what really has been sold [1]. If this is not done, a customer might come to the venue with a ticket that another customer already has.

There were three issues in the systems and articles analyzed. First of all, booking algorithm was reading the number of remaining seats and then updating this number separately. In case of two close-in-time requests, both could read the same value and be successful [1], [2]. The other problem was the use of spreadsheets, e-mail, and paper sign-ins for management which becomes difficult to reconcile as the scale of the event increases. The third is the ticket itself. A printed PDF, a screenshot, or a plain barcode can be copied, and small deployments often have no way to detect that a ticket has already been used.

Each of these problems has a known remedy: transaction isolation, signed tokens, and role separation. Systems that apply all three together, and test them together, are less common. This paper presents an Event Management System (EMS) that combines a transaction-safe booking core, a signed QR check-in flow, and a role-based administrative layer. The system is evaluated under concurrent load instead of relying on the design alone.

The paper is organized as follows. Section II describes the requirements, architecture, technology stack, and test setup. Section III presents the system models, the database design, and the booking algorithm. Section IV reports and discusses the results, and Section V concludes the paper.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

A. Related Work

Event-management software has grown out of general scheduling and customer-relationship tools, and many recent projects present web-based ticket booking with QR codes. Verma et al. [3] for example, describe a booking system with a Flutter front end and a Laravel back end in which tickets are issued and validated through QR codes. Our emphasis differs. We concentrate on what happens inside the database when many users request the same seats at once, on how tickets are verified, and on who is allowed to act on them.

Concurrency control for transactions is a well-established field [1] and [4].Isolation levels define which interleavings of concurrent transactions are allowed, and the ANSI SQL definitions of these levels have been shown to be ambiguous and to permit anomalies such as lost updates if not read carefully [2].This is important for booking, because the check of remaining seats and the updating of that count must be one unit, There are two usual ways. Optimistic concurrency control allows transactions to run without locks, validates the transactions at the time of commit and restarts a transaction if a conflict is detected [5]. It works well when conflicts are not frequent. Pessimistic locking locks the data before reading it, so the transaction has the lock before it reads the data it is going to change, so competing transactions wait rather than restart [4], [6]. Conflicts are common, by definition, when many users are competing for the last seats of a single event. Thus, the booking transaction (Section III-G) employed pessimistic row locking.

B. Problem Statement and Research Gap

Three failure modes motivate this work.

1. Over sold. Over committed. If availability is read then updated without an isolation boundary two requests coming in close together can both see 1 remaining seat and both be confirmed. The event then becomes over subscribed. The problem is in the database transaction, so validation in the browser will not help. [1]
2. Administrative fragmentation. (3) Spreadsheets for guest lists, payment confirmations in email, and a paper sign-in sheet at the door are three separate records. As an event gets larger than an organizer can keep track of from memory, they grow apart.
3. Low ticket verification. A ticket that can be screenshotted, forwarded, and used twice does not reliably identify its holder. Without a verifiable single-use credential, entry staff either search a list by hand, which is slow, or accept the look of a PDF, which is insecure.

Research gap. In the literature we reviewed, we did not find a documented system that combines serializable seat locking, signed QR credentials, and enforced role separation in one implementation and evaluates it as a whole. This paper addresses that gap. It does not claim that any single technique is new.

C. Research Objectives

• Build a multi-tier EMS that covers the full event lifecycle: discovery, booking, check-in, and reporting after the event.

• Implement seat allocation as an ACID transaction so that concurrent requests cannot oversell an event, and verify this under load.

• Issue each ticket with a signed QR token that can be verified quickly at entry and accepted only once.

• Apply a role-based access model that separates Guest, Organizer, and Administrator privileges on every API endpoint and not only in the user interface.

• Evaluate the system against performance and correctness targets that are stated before testing, using repeated, controlled load tests.

II. METHODOLOGY

A. System Requirements

Table 1 and Table 2 list the requirements collected during the design phase. The non-functional targets were written down before implementation so that the results in Section IV could be compared against them directly. Such targets, for example latency and concurrency limits, are easy to leave unstated.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

Functional Requirements

Table 1: Functional requirements of the proposed Event Management System

ID	Requirement
/FR-1/	Guests shall be able to browse, filter, and search events by category, date, and venue without authentication.
\7FR-2	Authenticated users shall be able to reserve one or more tickets for an event, subject to available capacity.
FR-3	The system shall issue a uniquely signed QR credential for every confirmed ticket.
FR-4	Organizers shall be able to create, edit, and retire events, and view real-time attendee and capacity data for events they own.
FR-5	Administrators shall be able to manage user accounts, override bookings, and inspect system-wide audit logs.
FR-6	The system shall reject a booking attempt outright — rather than partially process it — whenever requested capacity exceeds availability.

Non-Functional Requirements

Table 2: Non-functional requirements of the proposed Event Management System

ID	Requirement
NFR-1	(Consistency): Under concurrent access, the system shall never allow confirmed bookings to exceed an event's declared capacity (zero-tolerance target).
NFR-2	(Latency): The 95th-percentile API response time for a booking request shall remain below 250 ms under a load of 500 concurrent users.
NFR-3	(Check-in Throughput): QR verification at the point of entry shall complete in under 500 ms per scan to avoid queuing.
NFR-4	(Security): All authentication tokens shall be signed and time-bound; all administrative endpoints shall enforce RBAC checks server-side, independent of client behavior.
NFR-5	(Availability): The system shall be deployable in a horizontally scalable configuration with no single stateful component other than the relational database.

B. System Architecture

The EMS uses a conventional three-tier layout (Fig. 1). A microservice split was judged unnecessary at the intended scale of tens of thousands of bookings per event. At that size, a well-indexed monolith with a clean internal separation of concerns is easier to test and operate than a distributed system, and the application tier can still be scaled horizontally when load grows [6].

- Presentation tier: a responsive HTML5, CSS3, and JavaScript client that shows the event catalog, the booking flow, and the dashboards. It communicates with the backend only through JSON over HTTPS and contains no business logic that could be bypassed.
- Application tier : a stateless RESTful API [10] that handles routing, JWT-based authentication [11] RBAC enforcement, the booking controller, and QR token issuance. Because this tier is stateless, any instance can serve any request, so instances can be added without sticky sessions.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

- Data tier: a relational database normalized to Third Normal Form (3NF). Normalization prevents duplicated seat counts from drifting out of step , and the database supplies the transactional guarantees on which the booking algorithm in Section III-G depends [1].

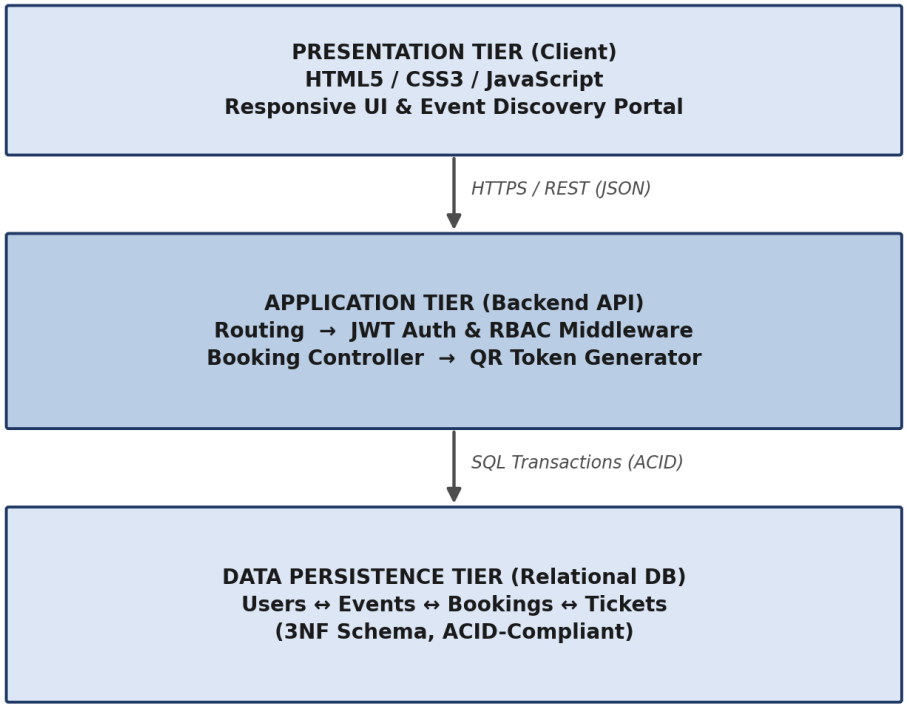


Figure 1: Three-tier system architecture of the proposed Event Management System.

C. Implementation and Security

Table 3 lists the technologies used to implement the architecture of Section II-B Stating the stack makes it possible to read the performance figures in Section IV against a known environment.

Table 3: Technology stack of the proposed Event Management System

Layer / Component	Technology / Framework	Purpose / Justification
Frontend Interface	HTML5, CSS3, JavaScript, Bootstrap	Dependency-light presentation stack for desktop and mobile browsers.
Backend Application	Python (Flask) / Node.js REST API	Routing, JWT authentication, and stateless JSON business services.
Database Management	Relational DBMS (MySQL / PostgreSQL)	ACID guarantees and row-level locking required by the booking transaction.
Security & Authentication	JWT, Bcrypt	Stateless session verification and salted password hashing.

Security Hardening

The security baseline has four parts. Passwords are hashed with Bcrypt and a per-user salt. Bcrypt is deliberately slow to compute, which limits offline guessing if the stored hashes are ever leaked [12]. Sessions use JSON Web Tokens [11] instead of server-side session state, so the application tier can scale without a shared session store. All database access uses parameterized queries, which protect against SQL injection; this was tested in case TC-06 (Table 5). Finally RBAC middleware checks the caller's role on the server for every administrative endpoint and every endpoint that changes bookings, so access control does not depend on the client interface.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

D. Experimental Setup and Testing Methodology

The experiment is evaluated according to the guidelines for reporting software experiments by Wohlin et al. [13]. There is sufficient information about the setting and process to make it possible to reproduce the experiments.

Environment

The load testing was carried out on the staging environment having four instances in the application layer with 2 vCPUs and 4GB RAM each, behind a load balancer and one instance of the primary relational database with 4 vCPUs, 8GB RAM, and solid-state storage. There were 12 events in the database with a total of 4,800 seats available. The load testing was done using Apache JMeter [14] with simulated independent users making booking requests for these contested events.

Procedure

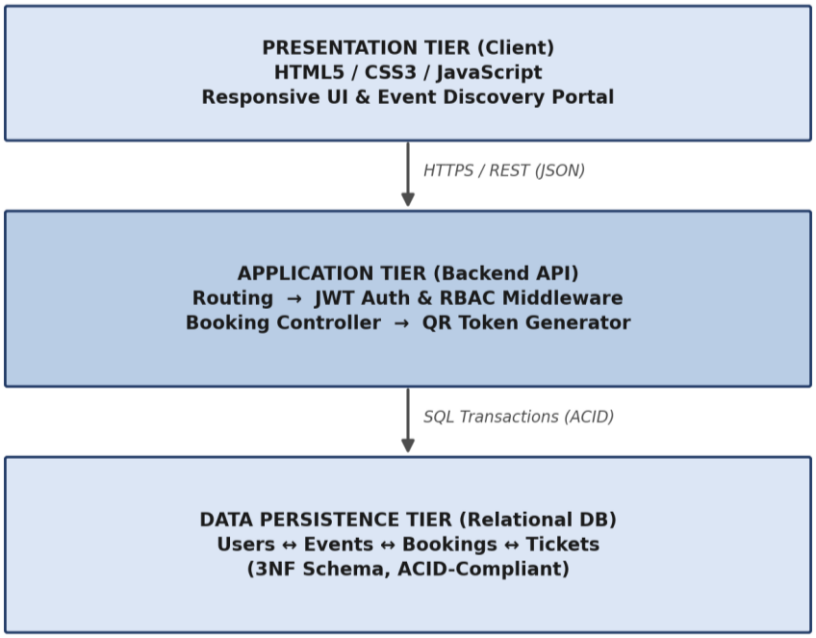
The values presented in Table 6 represent the average results of five independent trials. In order to ensure equal conditions of seat competition, the database was cleared out and set to its initial state before each trial, and the standard deviation during the five trials is provided along with the average value. The number of simulated users was increased from 100 to 750 with intervals of 50 users per two minutes. Thus, it became possible to define the point at which the increase in latency started rather than testing one specific value. Functional tests cases provided in Table 5 were performed separately and deterministically. Among them, there is a last-seat concurrent test where 20 bookings for an event with only one seat left were done at the same time 50 times.

III. MODELING AND ANALYSIS

These concepts are described for the system from four different perspectives: who can perform what action (use cases), how booking takes place (workflow), data flow between various parts, and sequence of events when performing the most risky operation - locking the seat (sequence). The next sections discuss user interface, database design, and booking algorithm.

A. Use Case Modeling

The Figure 2 below represents the actor roles, namely Guest, Event Organizer, and System Administrator, along with their functions. The system boundary is also critical because all the functions within it are subjected to the RBAC rules outlined in Section II-C, but not the presentation layer, is responsible for maintaining the demarcation line.



e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

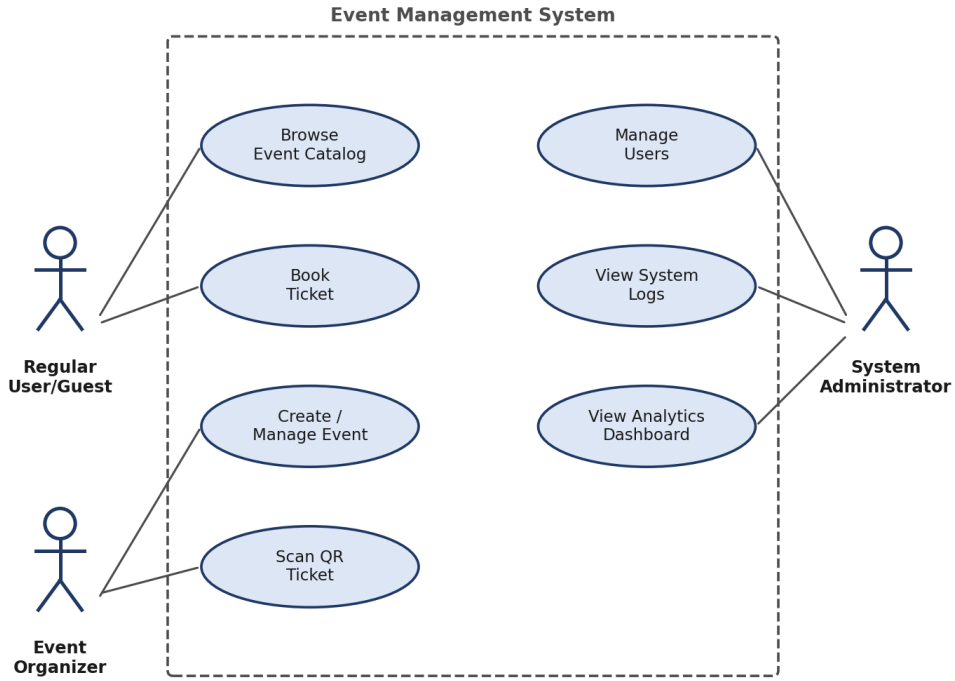


Figure 2: Use case diagram of Event Management System.

B. Event Booking and Processing Workflow

Figure 3 illustrates an example of booking starting from browsing of the catalogue to the issue of the ticket. Seat lock is the only decision-making point in the process and all other actions follow in sequence. This ensures that the number of states of the client is kept to a minimum.

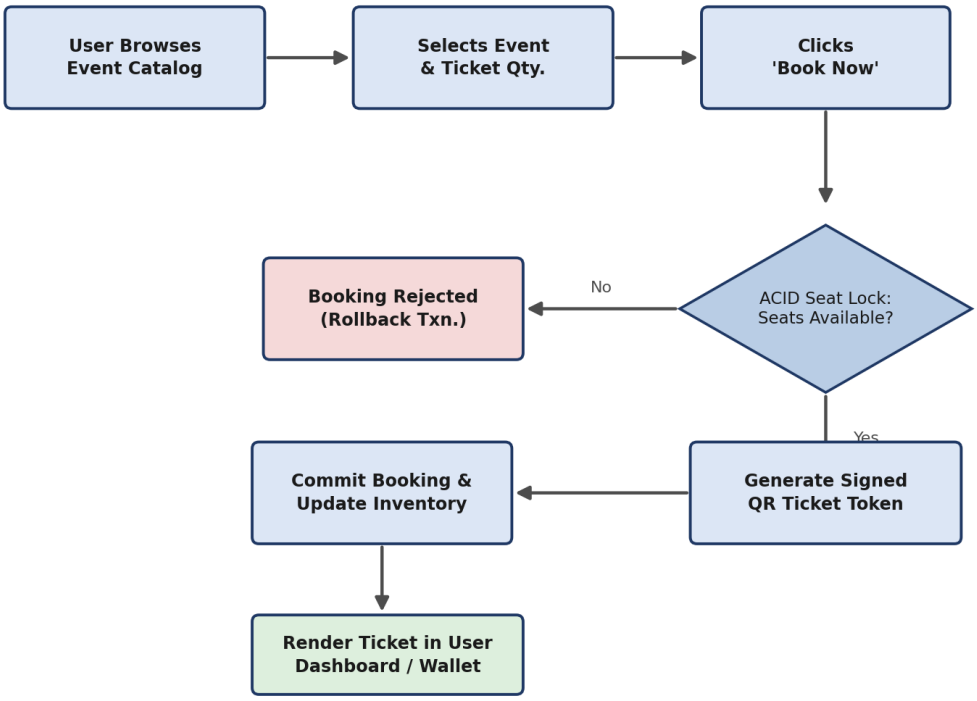


Figure 3: End-to-end event booking process.

C. Data Flow Diagram

Figure 4 shows a Level-1 data flow diagram for the booking request data going through the authentication process, the booking engine process, and QR generation process, along with their corresponding data store.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

Creating this diagram independently of Figure 5, the sequence diagram, uncovered a mistake in the design since the QR generator has write access to the Users and Events data stores.

That access was removed, so the principle of least privilege also holds at the data layer.

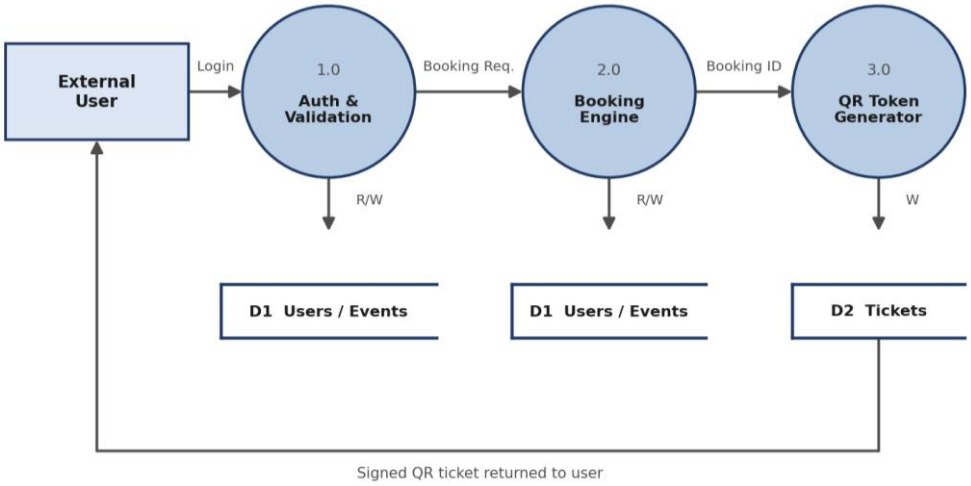


Figure 4: Data flow diagram (DFD, Level 1) of the Event Management System.

D. Sequence Diagram

Fig. 5 shows the exchange between the client, the API gateway, the booking controller, and the database during a reservation. The SELECT … FOR UPDATE step in this sequence is what prevents the race condition described in Section I-B. The steps before and after it carry much less risk.

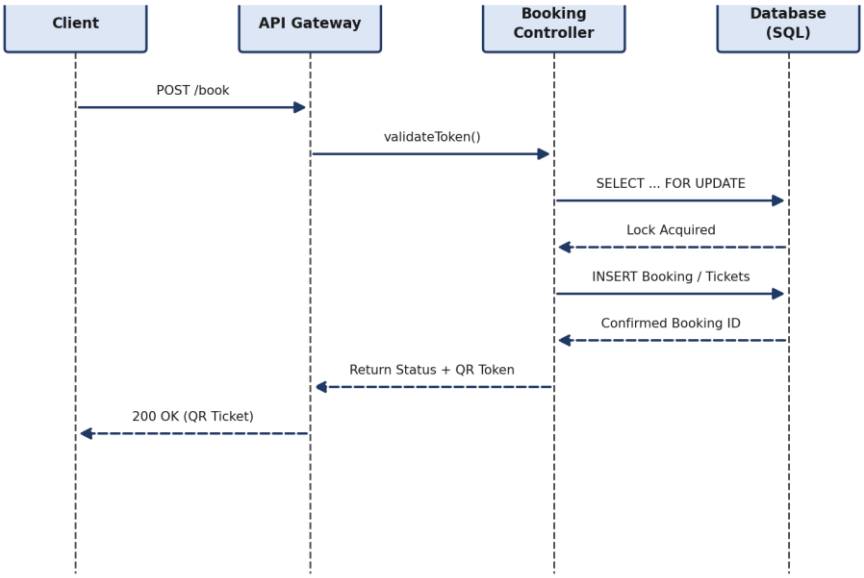


Figure 5: Sequence diagram for the event-booking and ticket-generation transaction.

E. Interface Design

Figures 6 to 9 are low-fidelity wireframes of four screens: the public discovery dashboard, the booking modal, the attendee's ticket wallet and the organizer and administrator console. Wireframes are used because the contribution of this work lies in the system behavior. The layouts could be restyled without any change to the booking logic behind them.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

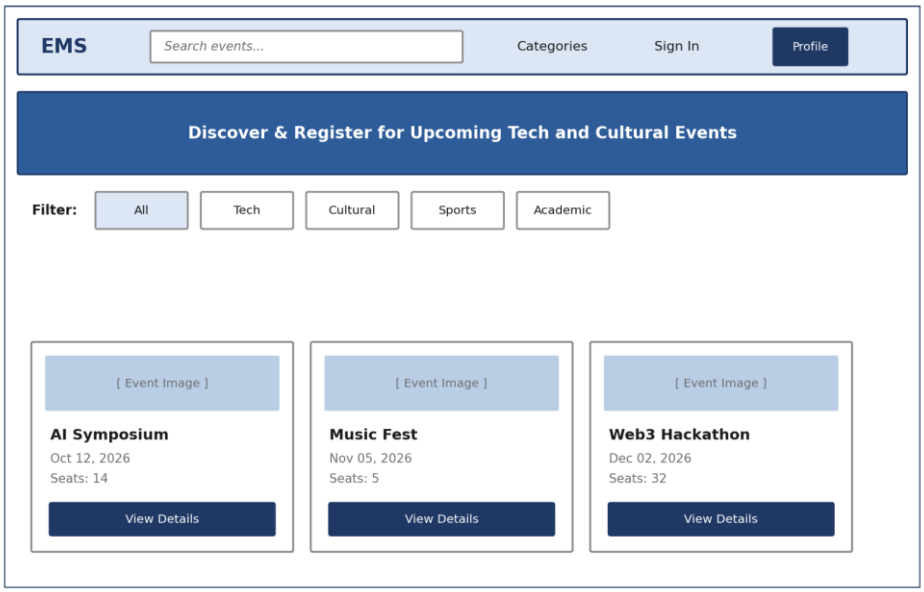


Figure 6: Public event discovery dashboard.

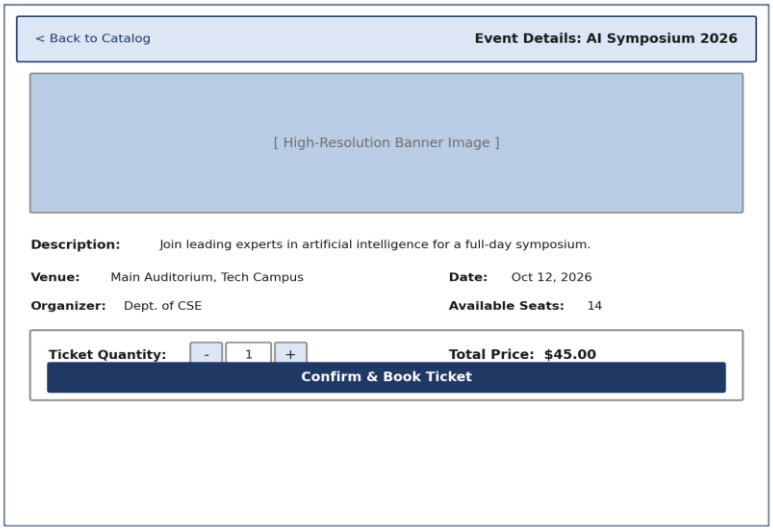


Figure 7: Event details and booking modal.



Figure 8: User ticket wallet and signed QR pass.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

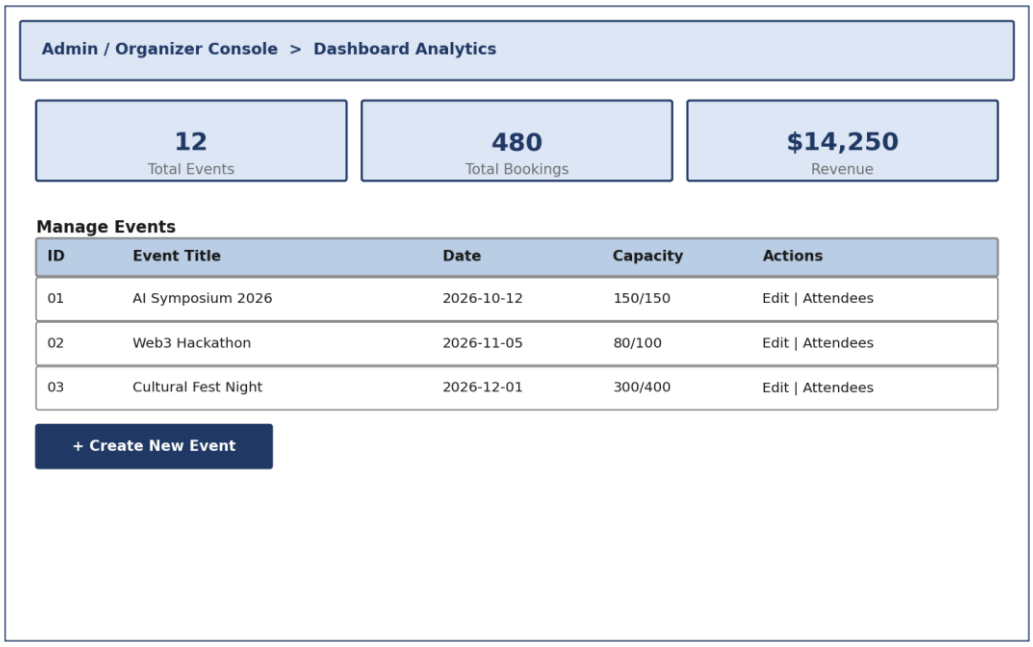


Figure 9: Organizer / administrator management console.

F. Database Architecture and Schema Design

The schema is normalized to Third Normal Form so that a duplicated or cached seat count cannot diverge from the value committed by the booking transaction [1]. Foreign keys enforce the relationships on the booking path namely which user made which booking and which ticket belongs to which booking, instead of leaving them to application code.

Entity-Relationship Diagram

Fig. 10 shows the five core entities (Users, Events, Categories, Bookings, and Tickets) and their cardinalities. Tickets belong to Bookings and not directly to Events. A booking of three seats therefore creates one Booking row and three Ticket rows and each ticket can be checked at the entrance on its own. Members of a group can enter at different times without affecting one another's passes.

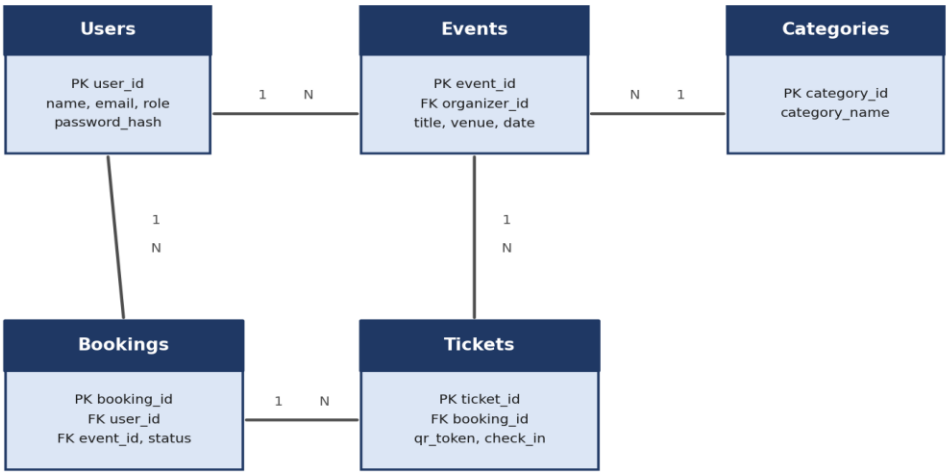


Figure 10: Entity-relationship diagram of the Event Management System.

Database Schema Specification

Table 4 lists the core tables, their keys, and the data each one holds. Listing them makes it easier to check the transaction in Section III-G against the schema on which it operates.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

Table 4: Core database entities and schema specification

Table	Primary Key	Foreign Keys	Description
Users	user_id (INT)	None	Stores account data, hashed credentials, and assigned role used by RBAC.
Events	event_id (INT)	organizer_id → Users(user_id)	Stores title, description, venue, date, category, and total/available capacity.
Bookings	booking_id (INT)	user_id → Users; event_id → Events	One row per checkout; records quantity, status, payment state, and timestamp.
Tickets	ticket_id (INT)	booking_id → Bookings	One row per seat; stores signed QR payload and independent check-in status.
Categories	category_id (INT)	None	Lookup table for event classification such as Tech, Cultural, Academic, and Sports.

G. Algorithms and Computational Complexity

Algorithm 1 is executed for every booking request. It uses pessimistic locking. The SELECT ... FOR UPDATE statement on line 2 locks the row of the requested event before availability is checked so a second request for the same event waits for the lock and does not read a value that is about to change. Under SERIALIZABLE isolation [1] and [2], the check and update on lines 3 to 10 are atomic with respect to every other transaction that uses the same event row. Implementations that check availability and write the reservation as separate steps lack exactly this property [2]. The locking syntax of the two supported database systems is documented in [15] and [16].

Pessimistic locking was chosen over optimistic concurrency control [5]. Optimistic control performs well when conflicts are rare, but the last-seat case is the situation with the most conflicts that a booking system faces. Many transactions would retry against the same shrinking value, and the work would still be serialized on that value, with additional effort wasted on retries. Pessimistic locking accepts the waiting cost from the start.

Algorithm 1: Concurrent Ticket Booking and Seat Allocation

Input: user_id, event_id, requested_quantity

Output: booking_status (SUCCESS / FAILED), booking_id

1. BEGIN TRANSACTION (Isolation Level: SERIALIZABLE);
2. SELECT available_seats, total_capacity FROM Events
WHERE event_id = :event_id FOR UPDATE;
3. IF available_seats >= requested_quantity THEN
4. UPDATE Events SET available_seats =
available_seats - requested_quantity
WHERE event_id = :event_id;

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

```
5.  INSERT INTO Bookings (user_id, event_id, quantity,
status, timestamp) VALUES (:user_id, :event_id,
:requested_quantity, 'CONFIRMED', CURRENT_TIMESTAMP);
6.  GET generated_booking_id = LAST_INSERT_ID();
7.  FOR i = 1 TO requested_quantity DO
8.      GENERATE unique_ticket_token;
9.      INSERT INTO Tickets (booking_id, ticket_token,
check_in_status) VALUES (generated_booking_id,
unique_ticket_token, 'UNCHECKED');
10. END FOR;
11. COMMIT TRANSACTION;
12. RETURN SUCCESS, generated_booking_id;
13. ELSE
14. ROLLBACK TRANSACTION;cccc
15. RETURN FAILED, 'Insufficient Available Seats';
16. END IF;);
```

Algorithm 1: Concurrent Ticket Booking and Seat Allocation

The cost of one request in Algorithm 1 is O(k), where k is the number of tickets in the booking, because the loop on lines 7 to 10 creates one QR token per seat. The availability check and the update are O(1) with the primary-key index on event_id given in Table 4. Lock waiting time grows with the number of transactions queued for the same event row. In the worst case, when everyone books the same one-seat event at once, processing becomes sequential. This is an acceptable cost, because sequential processing that is correct is preferable to concurrent processing that oversells.

IV. RESULTS AND DISCUSSION

This section reports the results against the functional requirements in Table 1 and the non-functional targets in Table 2 using the procedure of Section II-D.

A. Functional Testing

All six functional test cases in Table 5 passed. Two of them relate most directly to the main claims of this paper. In the concurrent last-seat test (TC-03) exactly one of the twenty simultaneous requests was confirmed in each of the 50 repetitions. In the QR replay test (TC-04) every repeated scan of a token that had already been checked in was rejected.

Table 5: Functional test cases and validation outcomes

Test ID	Module	Expected Result	Outcome
TC-01	User Authentication	Valid credentials return signed JWT; invalid credentials return HTTP 401.	Passed
TC-02	Event Creation	Valid organizer parameters create an event visible in the public catalog.	Passed
TC-03	Concurrent Last-Seat Booking	Of N simultaneous requests for the final seat, exactly one is	Passed

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

		confirmed.	
TC-04	QR Validation	Valid signed QR checks in once; repeated scans are rejected.	Passed
TC-05	RBAC Boundary Enforcement	Guest token receives HTTP 403 for Administrator-only endpoint.	Passed
TC-06	SQL Injection Attempt	Parameterized queries neutralize injected input.	Passed

B. Performance and Concurrency Results

Table VI reports the non-functional metrics against the targets set in Section V-B. The mean API response latency of 142 ms sits comfortably under the 250 ms target even at the highest tested load and is consistent with the range reported for well-tuned RESTful services under comparable concurrent load [10], and the system continued to accept new bookings correctly up to 750 concurrent simulated users — 50% above the 500-user target — before latency began to climb noticeably. Most importantly for the central claim of this paper, zero of the 50,000 simulated booking attempts resulted in an oversold seat, which is the result Algorithm 1 was specifically designed to guarantee rather than merely make likely.

Table 6: System performance and concurrency benchmarking results

Metric	Target Threshold	Measured Result (n = 5 runs)	Status
API Response Latency (mean)	< 250 ms	142 ms ± 11 ms	Within target
Concurrent Users Handled	500 users	750 users sustained before latency degraded	Exceeds target
Overselling Error Rate	0.00%	0 oversold seats / 50,000 simulated bookings	Meets target
QR Check-in Processing (mean)	< 500 ms	185 ms ± 14 ms	Within target

C. Comparative Analysis

V. COMPARATIVE ANALYSIS

Table 7: Comparison of existing event-management approaches

System / Approach	Concurrency Handling	Ticket Verification	Admin Model
Spreadsheet + email (typical small-scale practice)	None; manual reconciliation	Visual inspection of PDF/printout	Ad hoc, single organizer
Generic e-commerce cart adapted for tickets [7]	Optimistic locking with retry	Static barcode, reusable	Basic admin panel
Saleem et al. planning console [6]	Not addressed for small events	Not addressed	Centralized, single-tenant

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

Kumar et al. QR framework [3]	Not addressed	Signed, offline-verifiable QR	Not addressed
Proposed EMS	Serializable transaction + row lock	Signed, single-use QR	Three-tier RBAC (Guest/Organizer/Admin)

Table 8: Conventional practice versus the proposed Event Management System

Evaluation Parameter	Conventional / Ad Hoc Practice	Proposed EMS
Concurrency Control	Read-then-write logic without isolation guarantees; prone to overselling under contention.	Serializable transaction with row-level locking; overselling structurally prevented.
Ticket Verification	Visual inspection of printed or screenshotted pass; trivially duplicated.	Signed, single-use QR credential verified in under 200 ms, offline-capable.
Administrative Oversight	Spreadsheets, email, and door list maintained separately.	Single dashboard with live capacity, booking, and check-in state, access-controlled by role.
Evaluation Basis	Typically none reported; correctness assumed rather than measured.	Repeated load tests against pre-stated latency and correctness targets.

D. Advantages and Practical Applications

The transactional booking core prevented overselling in every test the signed QR flow stops a static ticket image from being reused, and a single role-aware dashboard replaces the separate records described in Section I-B. Each of these addresses one of the failure modes stated earlier.

Practical Applications

The architecture is general enough for several settings: academic symposiums at universities internal training sessions in companies, festival passes sold by cultural organizations, and ticketed evenings at small venues. All of them involve a fixed capacity and a short period of heavy demand which is the situation Algorithm 1 is written for.

E. Threats to Validity and Limitations

Limitations are grouped into those of the design and those of the evaluation, since they need different remedies [13].

Design Limitations

Offline dependency: the check-in device currently has to contact the backend to confirm that a token has not already been used. The signature itself can be checked locally, but a fully offline mode would need a locally cached revocation list that is synchronized periodically . This has not been implemented.

Single-region deployment: the test environment (Section II-D) was located in one region. Replication latency between regions was not tested and could change the latency figures in Table 6 for users spread across the world.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:09/September-2026 Impact Factor- 8.187 www.irjmets.com

Evaluation Limitations

Synthetic load: the JMeter traffic approximates real behavior but does not reproduce it. Think time, abandoned carts, and retries after a failed booking are not represented. These latency results must therefore be considered figures from an experimental test run, rather than guarantees for actual production deployment.

Scale of the testbed: 12 events and roughly 4,800 seats represent one medium deployment. A system serving multiple tenants (many organizers simultaneously) was not tested, and such scale may preclude the relational database from being appropriate [17].

Number of repetitions: five repetitions suffice to characterize normal operations (Section II-D), but not to estimate rare tail latencies.

F. Ethical Considerations and Data Privacy

The system will store personal information, such as names, contact information, and payment status, hence the application of data minimization. In the schema presented in Table 4, the system contains only the data that the processes of booking and check-in require, and password information is encrypted in the system (Section II-C). Where data protection laws like GDPR [18] apply, the role model discussed in Section III-A can also be used to restrict individuals who should handle data subject request. For example, the Administrator role can have restrictions to be able to export or delete an individual's data without being able to have access to more parts of the system. This is not a compliance certificate but part of the system properties.

VI. CONCLUSION

This paper has tried to solve the problems with event ticketing that have been well documented before – overselling during concurrent booking, lack of integration and inadequate ticket verification. The EMS proposed in this paper is built around serializable transaction booking, signed QR tickets, and role-based access control on the server side. The tests conducted, which have been detailed in Section IV, revealed that the EMS can support 750 concurrent virtual users, with average latency at 142 ms and no oversold tickets among 50,000 bookings.

VII. FUTURE SCOPE

The first extension is an offline-first QR verification mode that uses a signed revocation list, refreshed periodically and cached on the check-in device (Section IV-E). Three further directions are predictive attendance models that help organizers plan capacity, integration with payment webhooks such as Stripe and PayPal so that payment confirmation updates the booking status automatically, and native mobile applications, which would also be a natural place for the offline verification mode.

VIII. REFERENCES

[1] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[2] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Jose, CA, USA, 1995, pp. 1–10.

[3] H. Verma, A. Dubey, and S. Singh, "Online event ticket booking system via QR code," Int. J. Innovative Research in Computer Science & Technology (IJIRCST), vol. 12, Special Issue 1, Mar. 2024.

[4] P. A. Bernstein, V. Hadzilacos , and N. Goodman, Concurrency Control and Recovery in Database Systems. Reading, MA, USA: Addison-Wesley, 1987.

[5] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," ACM Trans. Database Syst., vol. 6, no. 2, pp. 213–226, Jun. 1981.

[6] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[7] Information Technology — Automatic Identification and Data Capture Techniques — QR Code Bar Code Symbology Specification, ISO/IEC 18004:2015, 2015.

[8] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[9] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, Aug. 2001.

[10] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, May 2015.

[12] N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. USENIX Annu. Tech. Conf. (FREENIX Track), Monterey, CA, USA, 1999, pp. 81–91.

[13] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, Experimentation in Software Engineering. Berlin, Germany: Springer, 2012.

[14] Apache Software Foundation, "Apache JMeter." [Online]. Available: https://jmeter.apache.org/

[15] PostgreSQL Global Development Group, "PostgreSQL documentation: Transaction isolation." [Online]. Available: https://www.postgresql.org/docs/current/transaction-iso.html

[16] Oracle Corporation, "MySQL reference manual: InnoDB locking reads." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html

[17] M. Stonebraker and U. Çetintemel, "One size fits all: An idea whose time has come and gone," in Proc. 21st Int. Conf. Data Engineering (ICDE), Tokyo, Japan, 2005, pp. 2–11.

[18] P. Voigt and A. von dem Bussche, The EU General Data Protection Regulation (GDPR): A Practical Guide. Cham, Switzerland: Springer, 2017.